WEEK 2 · DAY 4 · OVERVIEW

Decision Trees



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

A credit team has worked with a public file of 30,000 card accounts since Day 6.

The logistic model gives one equation, and a credit committee asks for the logic as plain rules, such as "two or more months late in September, and a September bill above 577 NT dollars".

A decision tree gives exactly that: a stack of yes or no questions fitted to the accounts.

The accounts are real, from Taiwan in 2005, so they describe consumer credit there and then, not US credit today.

The four person columns are never features, because a model that used them would sort people by them.

OBJECTIVES

Fit a tree and read it, see why a tree left alone memorises its accounts, and rein it in three ways: class weights, stopping it early, and cutting it back.

Choose each setting by cross-validation inside the training rows, compare the trees on validation rows, and say which columns a tree leans on.

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

Your Route Through Day 4

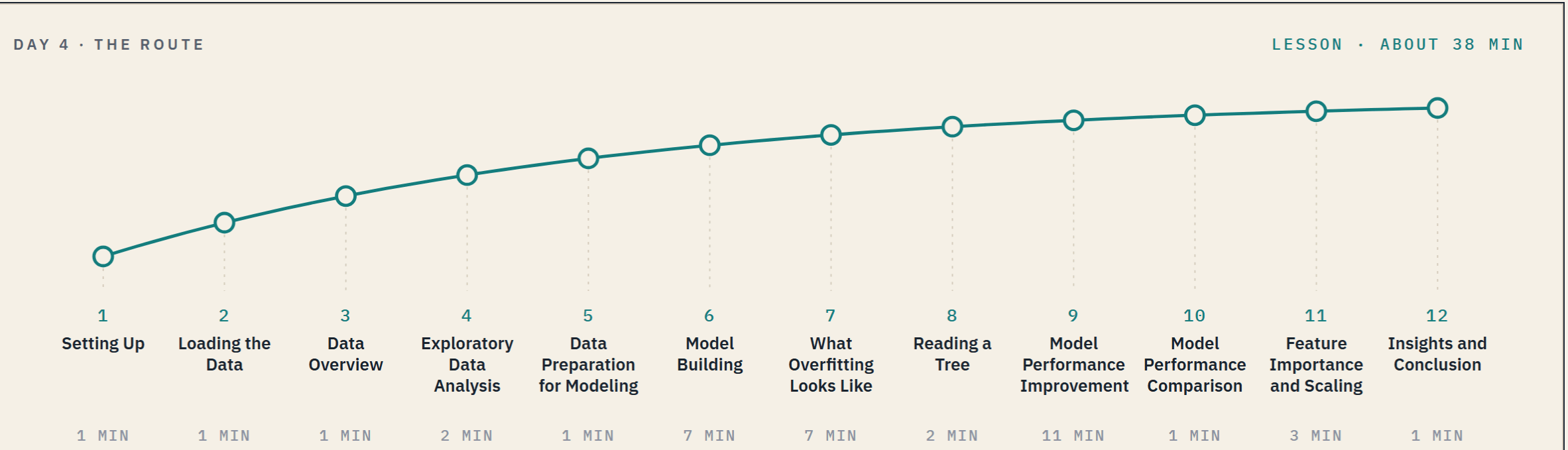


FIGURE 1 Twelve sections, in order, with about how long each takes.

Time. About 38 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

SEED, THREAD SETTINGS, AND WHERE THE DATA LIVES

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
    DATA = "https://raw.githubusercontent.com/lenamonj/python-fixed-income/main/data/"
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
```

Read the card file and pick 19 columns.

Loading the Data

THE 19 FEATURE COLUMNS

```
X = card.drop(columns=["ID", "SEX", "EDUCATION", "MARRIAGE", "AGE", TARGET])
y = card[TARGET]
FEATURES = list(X.columns)
print(len(FEATURES), "features")
```

```
19 features
```

WHAT EACH COLUMN HOLDS

- **LIMIT_BAL**: credit limit, NT dollars
- **PAY_0**, **PAY_2** to **PAY_6**: repayment status, September back to April 2005, where -1 is paid on time, 1 to 9 is months late, and -2 and 0 are undocumented (used as numbers, a modelling choice, as on Day 6)
- **BILL_AMT1** to **BILL_AMT6**: the bill, NT dollars, September back to April
- **PAY_AMT1** to **PAY_AMT6**: the amount paid, NT dollars, September back to April
- **default payment next month**: 1 if the account defaulted the following month, **the target**
- **ID**, **SEX**, **EDUCATION**, **MARRIAGE**, **AGE**: never used

Data Overview

SIZE AND DEFAULT SHARE

```
print(card.shape)
print("Defaults:", y.sum(), " Share:", round(y.mean() * 100, 2), "percent")
```

```
(30000, 25)
Defaults: 6636  Share: 22.12 percent
```

GAPS AND REPEATS

```
print("Missing values:", card.isnull().sum().sum())
print("Repeated rows:", card.drop(columns="ID").duplicated().sum())
```

```
Missing values: 0
Repeated rows: 35
```

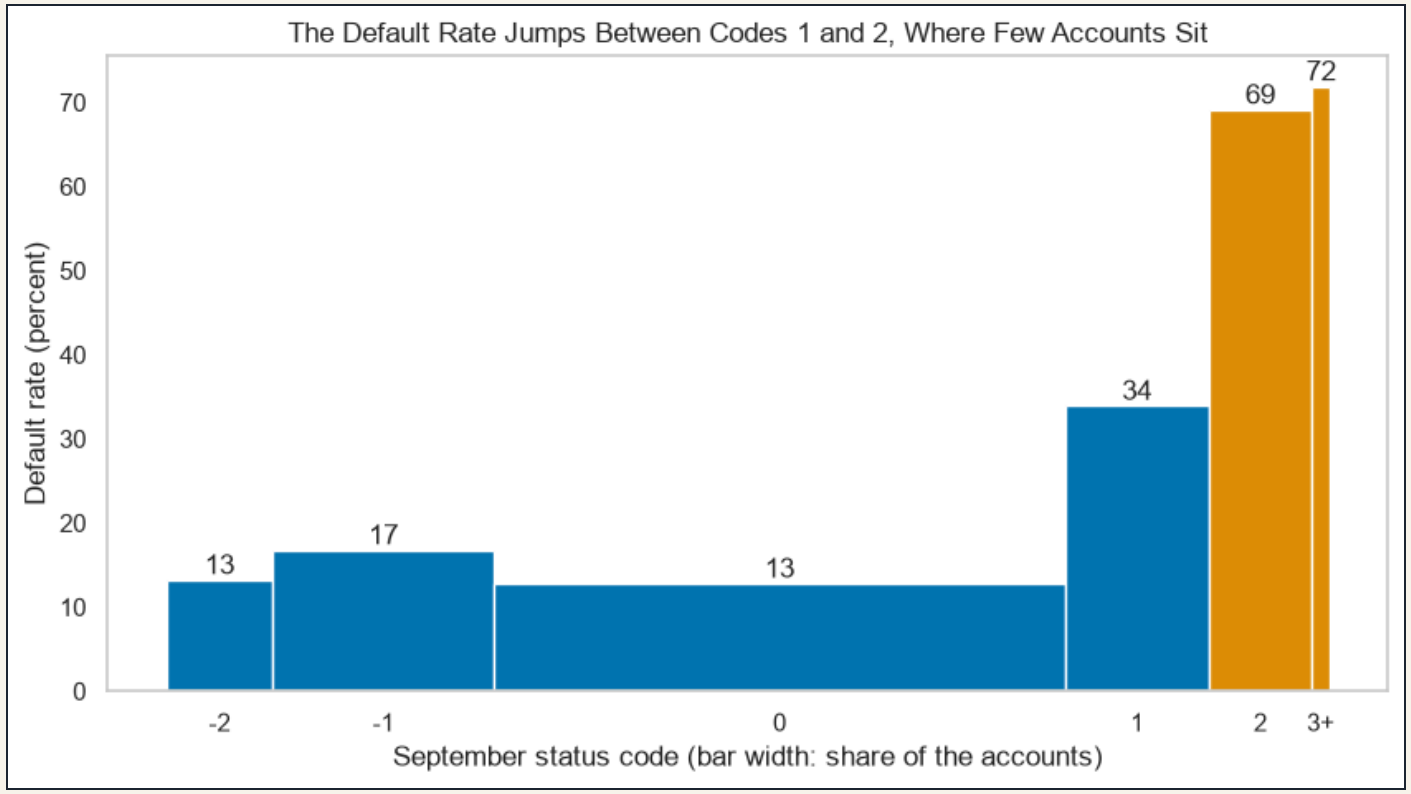
WHAT IT MEANS

Nothing is missing.

35 rows repeat another row's numbers under their own IDs: different accounts, kept.

The file is the one Days 6 to 8 used, complete and clean, so we look straight at the columns a tree will ask about.

Exploratory Data Analysis



ACCOUNTS AND DEFAULT RATE BY CODE

PAY_0	ACCOUNTS	DEFAULT RATE
-2	2759	0.1323
-1	5686	0.1678
0	14737	0.1281
1	3688	0.3395
2	2667	0.6914
3	463	0.7192

One column with a jump and one with a slide. A question of the form "is this value at most t?" can use both, and that is the question a tree asks.

Cut the accounts into training, validation and test.

Data Preparation for Modeling

TRAINING, VALIDATION AND TEST

```
# Day 6's split: a fifth of the accounts to test, then a quarter of the rest to validation, defaults spread evenly
X_rest, X_test, y_rest, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
X_train, X_valid, y_train, y_valid = train_test_split(X_rest, y_rest, test_size=0.25, stratify=y_rest, random_state=SEED)
print("Training:", len(X_train), " Validation:", len(X_valid), " Test:", len(X_test))
print("Default share:", round(y_train.mean(), 4), round(y_valid.mean(), 4), round(y_test.mean(), 4))
```

```
Training: 18000  Validation: 6000  Test: 6000
Default share: 0.2212 0.2212 0.2212
```

Training rows fit, validation rows report, and the test rows stay closed until Day 10.

Model Building

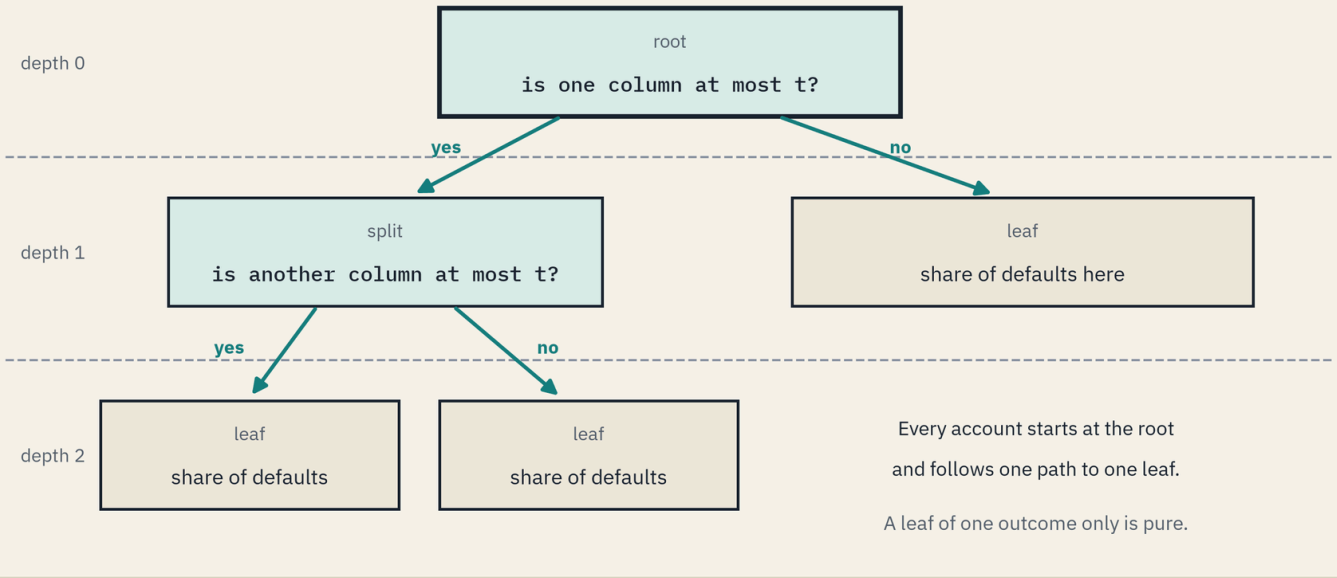


FIGURE 2 A tree two questions deep: the root and a split each ask a question, yes goes left, and every path ends in a leaf.

A TREE ALLOWED ONE QUESTION, AS TEXT

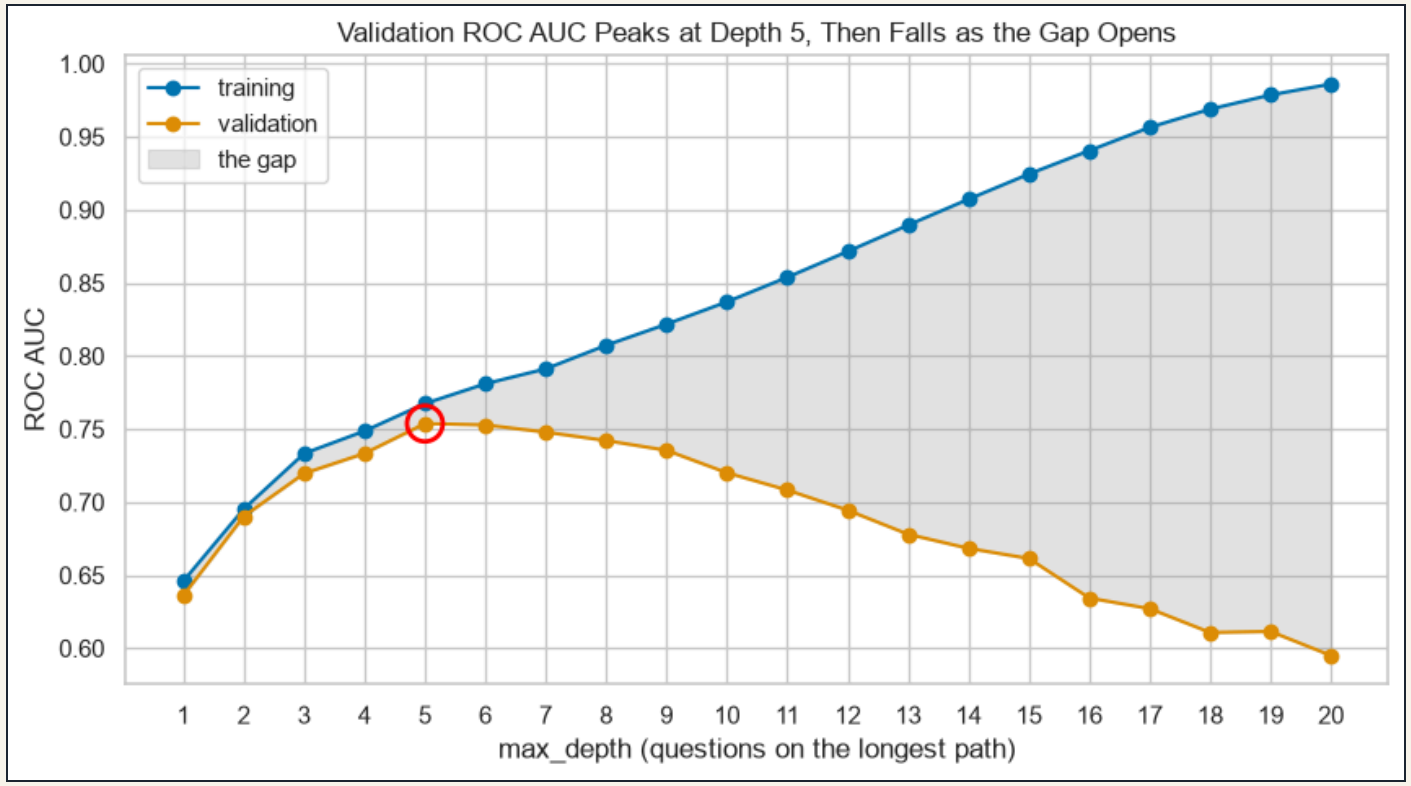
```
|--- PAY_0 <= 1.50
|   |--- weights: [13455.00, 2660.00] class: 0
|--- PAY_0 >  1.50
|   |--- weights: [563.00, 1322.00] class: 1
```

THE STUMP ON UNSEEN ACCOUNTS

	ROC AUC	RECALL	PRECISION
0	0.6361	0.3135	0.6831

Fitting a tree is this search, repeated inside each side. Nothing tells it when to stop.

What Overfitting Looks Like



THE FULL TREE ON THE ACCOUNTS IT WAS FITTED ON, THEN ON NEW ONES

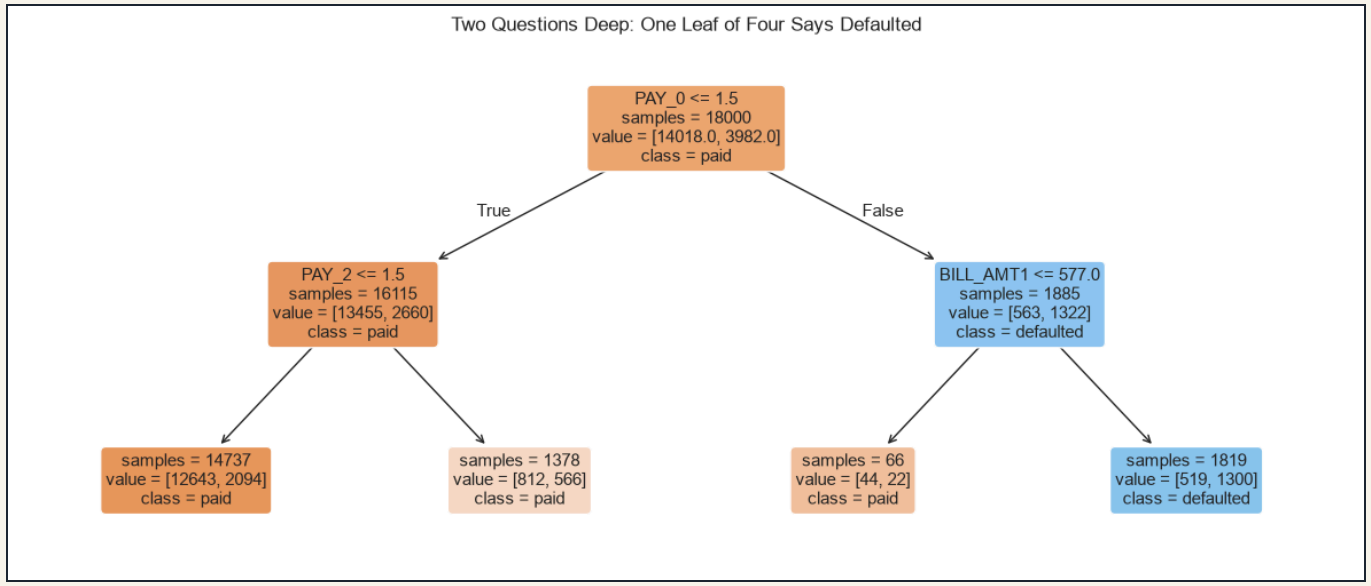
	ROC AUC	ACCURACY	RECALL
Training	0.9997	0.9913	0.9709
Validation	0.6161	0.7278	0.4032

READING THE CURVE

Validation ROC AUC peaks at depth 5 (0.7538), and the gap opens after it.

ROC AUC 0.9997 on the training accounts and 0.6161 on the validation accounts, below the stump's 0.6361.

Reading a Tree

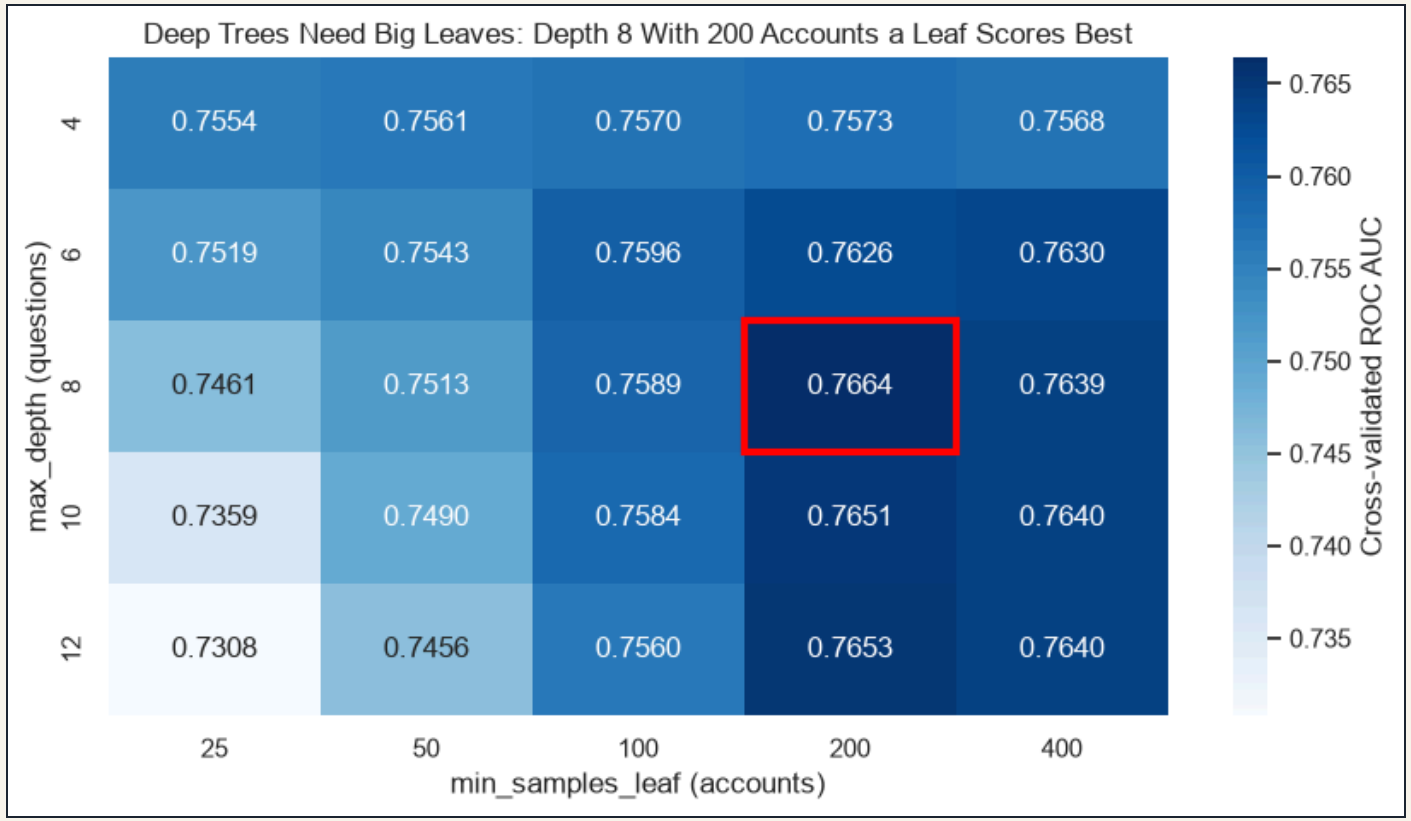


THE SAME KIND OF TREE AS RULES

```
|--- PAY_0 <= 1.50
|   |--- PAY_2 <= 1.50
|   |   |--- PAY_AMT2 <= 781.50
|   |   |   |--- weights: [2653.00, 763.00] class: 0
|   |   |--- PAY_AMT2 >  781.50
|   |   |   |--- weights: [9990.00, 1331.00] class: 0
|   |--- PAY_2 >  1.50
|   |   |--- PAY_5 <= 1.00
|   |   |   |--- weights: [605.00, 348.00] class: 0
```

Each rule describes how this file splits, never a rule for lending.

Model Performance Improvement



THE BEST PAIR OF SETTINGS, AND ITS SCORE

```
{'max_depth': 8, 'min_samples_leaf': 200} 0.7664
```

WHY CROSS-VALIDATION

Choosing it on the validation rows would spend them, so we choose inside the training rows by cross-validation.

In the 400 column, from depth 6 down, the score barely moves (0.7630 to 0.7640): big leaves stop the tree on their own.

Line up four trees on training and validation.

Model Performance Comparison

VALIDATION SCORES

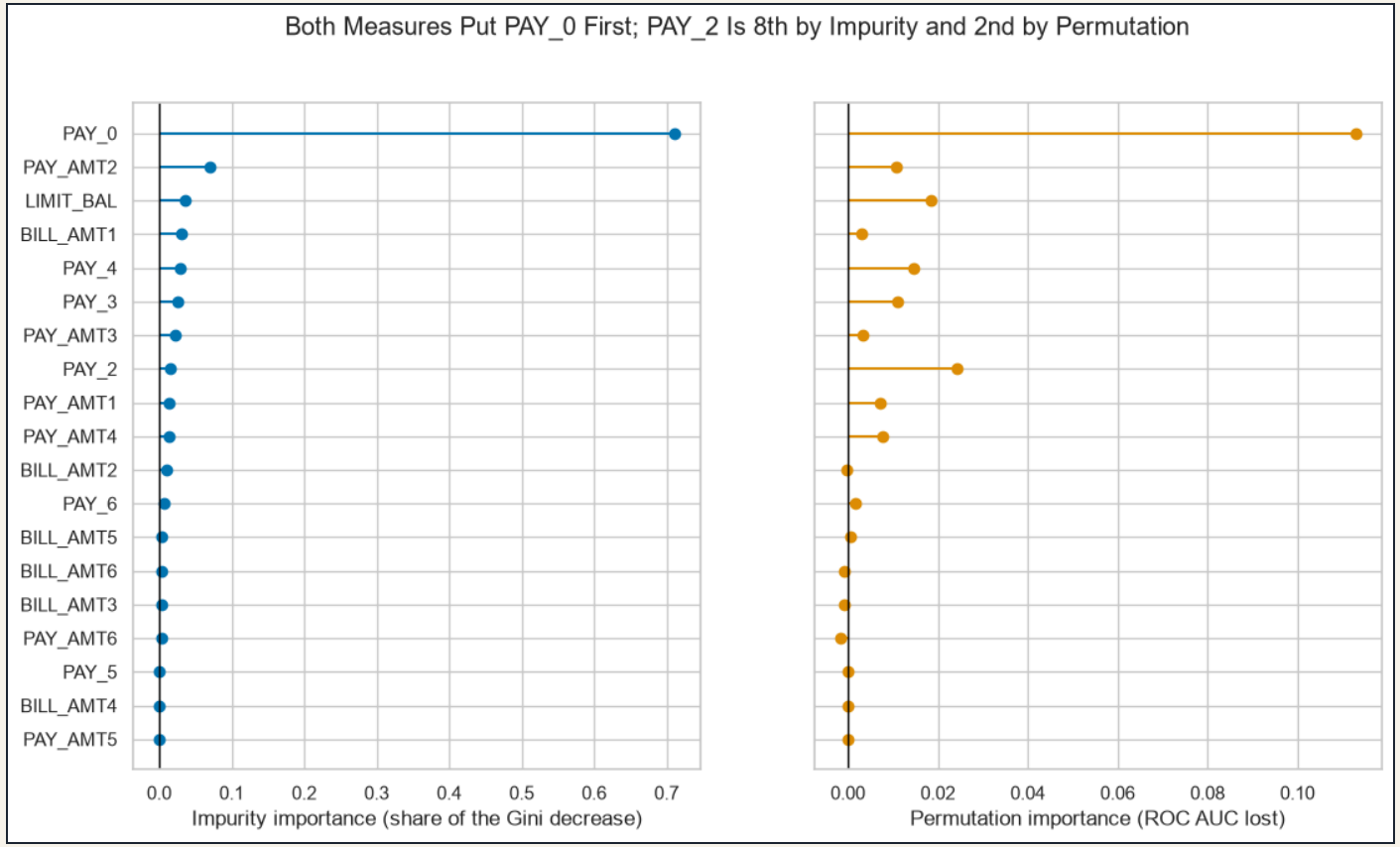
	FULL	PRE-PRUNED	POST-PRUNED
ROC AUC	0.6161	0.7633	0.7627
Accuracy	0.7278	0.7603	0.7683
Recall	0.4032	0.6149	0.5840
Precision	0.3888	0.4682	0.4805
F1	0.3959	0.5316	0.5272

WHAT THE VALIDATION ROWS CAN SAY

Choosing settings and comparing trees on the validation rows spends some of their honesty, which is why Day 10 opens the test rows once.

We carry the pre-pruned tree into importance, and Day 10 compares a pruned tree with the logistic model.

Feature Importance and Scaling



BOTH MEASURES, TOP EIGHT COLUMNS

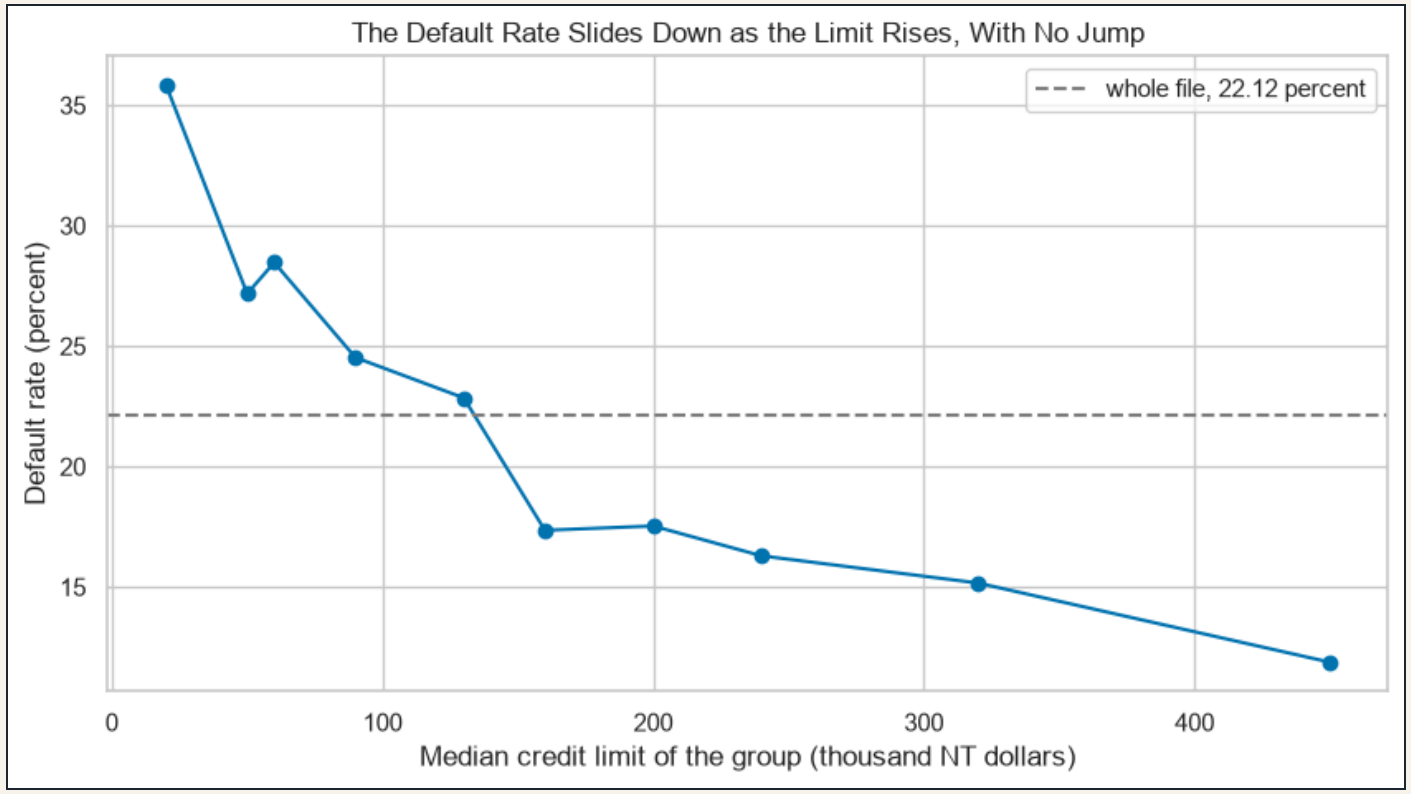
	IMPURITY	PERMUTATION
PAY_0	0.7115	0.1130
PAY_AMT2	0.0705	0.0108
LIMIT_BAL	0.0367	0.0185
BILL_AMT1	0.0303	0.0030
PAY_4	0.0293	0.0145
PAY_3	0.0263	0.0110

SCALING DOES NOT CHANGE A TREE

A split compares a value with a threshold, and scaling keeps every value's order, so the tree asks the same questions in new units.

PAY_2 is 8th by impurity and 2nd by permutation: its questions make little of the Gini decrease, yet shuffling it costs the second most on unseen accounts.

Insights and Conclusion



ONE COLUMN WITH A SLIDE

The group with the lowest limits defaults at 36 percent and the group with the highest at 12 percent.

The rate slides down across the groups, with no single step.

A tree is worth its rules only if they hold on accounts it has never seen; Day 10 opens the test rows once.

You Can Now

- Fit a decision tree, print its rules, and draw it.
- Say how a tree picks a question, with Gini impurity.
- Read a validation curve and a tree's map, and spot a tree that memorises.
- Use class weights, and prune a tree early or late, choosing its settings by cross-validation.
- Read impurity and permutation importance, and say what they do not mean.

NEXT
WEEK 2 · DAY 5

Case Study: Credit Card Default

Day 10 runs a logistic model against a pruned tree and opens the test rows once.